Gradient Bitumen

Conversational Sales Agent

Build & Vibe-Coding Plan for Claude Code

An end-to-end plan to build an AI agent that takes natural-language input (on the utility bar or the Account record page) and drives the **Account** → **Opportunity** → **Order** → **Loading Advice** → **Approval** process — validating required fields, asking for what is missing, confirming before it writes, and surfacing insight about the records along the way.

Target org	salesgvgr — PartialSB sandbox
Build tool	Claude Code + Salesforce CLI (sf)
Working directory	/Users/marendavid/Documents/gradient-bitumen-agent
Front-end	Agentforce agent (utility bar + Account page) — LWC chat fallback documented
Agent "tools"	Apex Invocable Action library (testable, deployable)

1. Objective & Scope

Deliver a conversational agent inside the Gradient Bitumen Salesforce app that lets a sales user describe what they want in plain language and have the system create the right records and route them for approval — without the user clicking through the standard screens.

In scope

- Agent reachable two ways: **(a)** pinned to the **utility bar** (works from anywhere), and **(b)** on the **Account record page**.
- **Context resolution:** on the Account page the agent uses the page *recordId*; from the utility bar it **asks for the Account name** and resolves it (with disambiguation if more than one match).
- Guided creation along the chain: **Opportunity** → **Order** → **Loading Advice** (*Commercial (Sale)* or *Stock Transfer (Internal)*) and the matching **Loading Advice Line**.
- **Auto-submit** the Loading Advice for approval once created.
- **Missing-field handling:** if a required field is absent, the agent asks for it rather than failing.
- **Confirm-before-write:** the agent shows a summary of what it will create and waits for explicit user confirmation before committing any record.
- **Insight:** the agent can answer questions about the objects (e.g. open opportunities, order status, recent loading advices, what fields a Loading Advice needs).

Out of scope (for the first build)

- Re-designing the existing approval routing, PDF templates, or notification logic — the agent **reuses** what already exists.
- Editing or deleting records via the agent (create + read only in v1; updates can be a fast-follow).
- Mobile-specific UX tuning beyond what Agentforce provides out of the box.

2. What Already Exists (and gets reused)

The org already implements the manual path the agent will automate. The plan deliberately wraps and reuses these assets instead of rebuilding them.

Existing asset	Type	Role for the agent
Loading Advice – Submit for Approval	Screen Flow	Source of truth for the approval logic (routing, PDF, notification, email). Reused via a headless sibling — see §6.
LoadingAdvicePDFGenerator	Apex (invocable)	Generates the internal LA PDF; already called by the flow.
LA_Build_Email_Attachments	Subflow	Assembles the PDF + LPOs into the approval email.
GenerateStockTransferPDF / StockTransfer / Dispatch / PendingLogistics PDF classes	Apex	Document generation already present; agent does not duplicate these.
Approvver_Availability__c	Custom object	Drives Level-1 primary vs backup approver routing.
Create Loading Advice (Type → fields)	Guided UI	The field set the agent must collect; mirrored in the agent's create action — see §4 / Appendix B.

Key reuse decision

The current **Submit for Approval** flow is a **screen flow** (it shows Error and Success screens). A screen flow cannot be launched unattended by an agent. We add one **autolaunched** flow that performs the same record update + PDF + notification + email steps without screens, and the agent calls that. The existing screen flow stays untouched for manual users.

3. Recommended Architecture

Two layers. The **conversation layer** understands the user and orchestrates. The **action layer** is deterministic Apex that validates and writes records. Keeping the writing logic in tested Apex (not in prompts) is what makes the build safe to vibe-code and easy to certify with required test coverage.

Layer A — Conversation / orchestration (recommended: Agentforce)

- An **Agentforce agent** provides the natural-language understanding, the utility-bar & record-page surfaces, conversational memory, and the "give me insight" answers for free.
- The agent is configured with **custom actions** that map 1:1 to the Apex invocable methods in Layer B.
- Agent **instructions/topics** encode the rules: resolve the account, walk the chain in order, ask for any missing required field, **always preview & get a yes before committing**.

> **Honest note on Agentforce**
> Claude Code can author and deploy all Apex, the GenAiFunction/GenAiPlugin metadata, and the autolaunched flow. **Final agent assembly and activation (topics, instructions, action wiring, testing in Agent Builder) is partly click-based in the org** and may require Agentforce licenses + Einstein generative AI enabled. The plan calls this out as a manual checkpoint so there are no surprises.

Layer B — Action library (Apex invocable, fully vibe-coded)

Each action is an `@InvocableMethod` with a typed request/response wrapper. They are pure, unit-tested, and reusable by either front end. See §4 for the catalogue.

Fallback front-end — custom LWC chat (if Agentforce is not licensed)

- A Lightning Web Component chat panel on the utility bar and Account page (`@api recordId` gives Account context automatically).
- An Apex controller calls an LLM through a **Named Credential** (Salesforce Models API, or Anthropic API) and uses tool/function-calling to invoke the same Layer-B actions.
- More code, but zero dependency on Agentforce licensing and full control of the confirm-before-write UX.

Flow of a request

1. User: "Create a commercial loading advice for A A Fugu & Sons for 30 MT of bitumen."
2. Agent resolves the Account (recordId on page, or by name from the utility bar).
3. Agent gathers context, walks the chain, and calls the *preview* mode of each create action.
4. Any required field missing → agent asks a targeted question and waits.
5. Agent shows a summary; user confirms.
6. Agent calls *commit* actions → Opportunity, Order, Loading Advice + Line are created.
7. Agent calls the headless approval action → PDF + notification + email fire.
8. Agent replies with the created record links and a one-line status.

4. Agent Action Library (the "tools")

Build these as separate Apex classes, each with its own test class. Create actions follow a **two-phase pattern**: a *preview* call returns what would be created plus any missing required fields; a *commit* call performs the insert. This is what powers "ask for missing fields" and "confirm before creating" without trusting the LLM to remember the rules.

Action	Purpose	Key inputs → output
Agent_ResolveAccount	Find the account by name or id; disambiguate.	name / accountId → matched Account(s) + summary, or 'needs clarification'
Agent_GetAccountInsight	Read-only insight for the agent's answers.	accountId → open opps, recent orders, recent loading advices, credit terms
Agent_DescribeObject	Explain an object & its required fields.	objectName → required/optional fields, picklist values
Agent_PreviewOpportunity / Commit	Validate + create Opportunity.	accountId, name, stage, amount, closeDate, businessUnit → missing[] or new Opp
Agent_PreviewOrder / Commit	Validate + create Order from an Opportunity.	opportunityId, startDate, quantity → missing[] or new Order
Agent_PreviewLoadingAdvice / Commit	Validate + create LA + LA Line.	orderId, loadType, product, qtyMT, price, customerType, location, … → missing[] or new LA
Agent_SubmitLAForApproval	Auto-submit via the headless flow.	loadingAdviceId → submitted status + approver routed

Standard response shape

```
public class AgentActionResult {
  @InvocableVariable public Boolean success;
  @InvocableVariable public String  status;       // OK | NEEDS_INPUT | NEEDS_CONFIRM | ERROR
  @InvocableVariable public String  recordId;      // populated on commit
  @InvocableVariable public String  recordUrl;
  @InvocableVariable public List<String> missingFields;  // drives the follow-up question
  @InvocableVariable public String  summaryJson;   // what was/ would be created (for the conf
irm step)
  @InvocableVariable public String  message;       // human-readable line for the agent
}
```

Why two-phase + structured results

The agent never invents field rules. Preview returns the authoritative *missingFields* list straight from the org's metadata and validation, so the model only has to ask the question. Commit refuses to run unless the caller passes a confirmation token, guaranteeing nothing is written without an explicit user "yes".

5. Data Model Reference

The chain and relationships the agent traverses. **Field API names must be confirmed against the org** during Phase 1 — the list below is drawn from the flow metadata and Schema Builder and is a starting point, not gospel.

Object	Links to	Relationship
Account	—	Top of the chain (customer)
Opportunity	Account	Lookup to Account
Order	Opportunity, Account	Lookup to Opportunity (and Account)
Loading_Advice__c	Order	**Master-Detail** to Order
Loading_Advice_Line__c	Loading_Advice__c	**Master-Detail** to Loading Advice

Loading Advice — fields the agent touches (confirm in org)

- `Approval_Status_DKL__c` (e.g. "Pending 1st Approver"), `Status_DKL__c` (e.g. "Draft").
- `SubmittedOn_DKL__c`, `Submittedby_DKL__c`, lookups to `Opportunity` / `Account`, plus approver fields.
- Type of Load is captured at creation: **Stock Transfer (Internal)** vs **Commercial (Sale)**.

Loading Advice Line — required vs optional (from the Create LA screen)

Required	Optional
Customer, Customer Type, Location, Product, Quantity (MT), Price (N/MT)	LPO Number, Delivery Mode, Currency, Transportation (N/MT), Payment Status, Plant Location, Payment Term, Heating Charge (N/TRK), Sprayer Rate (N/day), No. of Sprayer days

These required fields are exactly what the agent must collect before it can commit a Loading Advice; anything the user did not supply becomes a follow-up question.

6. Build Phases (the vibe-coding roadmap)

Each phase ends in something deployed and verified in the sandbox. Vibe-code one phase at a time, run the tests, deploy, eyeball it, then move on.

Phase 0 — Connect & orient

1. Authenticate Claude Code's `sf` CLI to the PartialSB sandbox (web login).
2. Create the working directory and initialise an SFDX project.
3. Retrieve metadata for the five objects, the Submit-for-Approval flow, and the LA/PDF Apex classes so the build is grounded in the **real** schema, not assumptions.

Phase 1 — Read-only foundation

1. Build **Agent_ResolveAccount**, **Agent_GetAccountInsight**, **Agent_DescribeObject** with tests.
2. Confirm every field API name retrieved in Phase 0; fix the data-model reference.
3. Deploy; verify the agent (or a quick anonymous-apex harness) can resolve an account and describe a Loading Advice.

Phase 2 — Guided creation (preview/commit)

1. Build the preview + commit pair for Opportunity, then Order, then Loading Advice + Line.
2. Enforce the confirmation token on every commit; enforce required-field validation on every preview.
3. Tests must cover: all-fields-supplied happy path, each missing-field branch, and commit-without-confirmation rejection.

Phase 3 — Headless auto-approval

1. Create the autolaunched flow that mirrors *Loading Advice – Submit for Approval* minus the screens (reusing `LoadingAdvicePDFGenerator` + `LA_Build_Email_Attachments`).
2. Wrap it in **Agent_SubmitLAForApproval**; verify PDF, notification, and email all fire.

Phase 4 — Conversation layer

1. **Agentforce path:** create the agent, add the actions as custom agent actions, write the topic instructions (resolve → walk chain → ask for missing → confirm → commit → submit → report), test in Agent Builder.
2. **LWC fallback:** build the chat LWC + Apex LLM controller (Named Credential) with function-calling to the same actions.
3. Place on the utility bar and the Account record page; wire record-context passing.

Phase 5 — Hardening & handover

1. End-to-end test from both surfaces; confirm "ask for account name" (utility) vs "use recordId" (Account page).
2. Confirm org-wide test coverage ≥ 75%; security review (FLS/sharing) on every action.
3. Document the manual Agent Builder steps and any licensing prerequisites for the client.

7. Testing, Security & Deployment

- Every Apex action ships with a test class; target ≥ 90% on new code so the org stays above the 75% deploy gate.
- Run against the sandbox only; never auto-deploy to production from the agent build.
- Validate **field-level security and sharing** for each action — the legacy flow runs *System Mode Without Sharing*; decide deliberately whether the agent actions should run in user or system context.
- Treat the confirmation gate as a tested invariant: a commit without a valid confirmation token must throw, with a test proving it.

8. Risks, Assumptions & Open Questions

Item	Type	Note / question for the client
Agentforce licensing & Einstein GA enabled	Assumption	If unavailable, switch to the LWC fallback front end (same actions).
Field API names	Risk	Confirmed in Phase 1 from the org; the reference list here is provisional.
Auto-submit = headless flow	Decision	OK to add an autolaunched sibling and leave the screen flow for manual users?
Order ↔ Opportunity required fields	Open Q	Which Order fields are truly mandatory at creation (quantity, start date)?
LA Type → different PDF	Open Q	Commercial vs Stock Transfer use different PDF classes — confirm which the agent should trigger per type.
LLM data handling (LWC path)	Risk	If using an external LLM, confirm what record data may leave the org and via which Named Credential.
Update/delete via agent	Scope	Confirmed out of v1?

Appendix A — Claude Code Kick-off Prompt

Paste this as the first message to Claude Code in a new session. It tells Claude Code to authenticate first, set up the workspace, learn the real schema, and build phase-by-phase with tests — and to stop and ask rather than guess.

```
You are a senior Salesforce engineer building a conversational sales agent for the
"Gradient Bitumen" org. We are vibe-coding this together, phase by phase. Do not rush
ahead: finish a phase, run the tests, deploy to the sandbox, show me the result, then ask
before starting the next phase. If you are ever unsure about a field name, an object, or a
requirement, STOP and ask me or query the org -- never guess.

=== STEP 0: AUTHENTICATE FIRST ===
Before anything else, log in to the sandbox and confirm the connection:
  sf org login web --alias gb-partialsb \
     --instance-url https://salesgvgr--partialsb.sandbox.my.salesforce.com
Then run `sf org display --target-org gb-partialsb` and show me the result so I can confirm
we are pointed at the right org. Do not proceed until I confirm.

=== STEP 1: WORKSPACE ===
Create and initialise the project here:
  /Users/marendavid/Documents/gradient-bitumen-agent
Run `sf project generate`, set the default org to gb-partialsb, and commit an initial git
state.

=== STEP 2: LEARN THE REAL SCHEMA (do not assume) ===
Retrieve and read the metadata for:
  - Objects: Account, Opportunity, Order, Loading_Advice__c, Loading_Advice_Line__c
    (capture EVERY field API name, type, required flag, picklist values, relationships)
  - The flow "Loading Advice - Submit for Approval"
  - Apex: LoadingAdvicePDFGenerator, LoadingAdvicePDFExtension, GenerateStockTransferPDF,
    and the subflow LA_Build_Email_Attachments
Write what you found into docs/schema.md and confirm the relationships:
Account -> Opportunity (lookup), Order -> Opportunity (lookup), Loading_Advice__c is
MASTER-DETAIL to Order, Loading_Advice_Line__c is MASTER-DETAIL to Loading_Advice__c.

=== WHAT WE ARE BUILDING ===
An agent that takes natural-language input and drives:
  Account -> Opportunity -> Order -> Loading Advice (Commercial OR Stock Transfer)
  -> auto-submit for approval.
It runs in two places: the utility bar (anywhere) and the Account record page.
  - On the Account page: use the page recordId as the customer.
  - On the utility bar: ASK the user for the Account name, then resolve it
    (if multiple matches, ask which one).
Rules the agent must always follow:
  - If a REQUIRED field is missing, ASK the user for it -- do not fail or invent a value.
  - Before creating ANY record, show a summary and get an explicit "yes" to confirm.
  - Be able to give insight about these objects (open opps, order status, recent loading
    advices, what fields a Loading Advice needs, etc.).

=== ARCHITECTURE ===
Front end: an Agentforce agent (preferred). Build the brains as a library of Apex
@InvocableMethod actions that the agent calls as tools, each with its own test class.
Use a two-phase pattern for every create: a PREVIEW method that returns missing required
fields, and a COMMIT method that refuses to run without a confirmation token.
If Agentforce is not licensed in this org, tell me and we will switch the front end to a
custom LWC chat panel + Apex controller calling an LLM via a Named Credential -- same Apex
actions either way.

Standard result wrapper for every action:
  success(Boolean), status(OK|NEEDS_INPUT|NEEDS_CONFIRM|ERROR), recordId, recordUrl,
```

```
  missingFields(List<String>), summaryJson, message.

Actions to build (each + test class):
  Agent_ResolveAccount, Agent_GetAccountInsight, Agent_DescribeObject,
  Agent_PreviewOpportunity / Agent_CommitOpportunity,
  Agent_PreviewOrder / Agent_CommitOrder,
  Agent_PreviewLoadingAdvice / Agent_CommitLoadingAdvice,
  Agent_SubmitLAForApproval.

=== AUTO-APPROVAL ===
The existing "Loading Advice - Submit for Approval" flow is a SCREEN flow and cannot run
unattended. Create an AUTOLAUNCHED flow that performs the same steps (record update + PDF
via LoadingAdvicePDFGenerator + custom notification + email via LA_Build_Email_Attachments)
WITHOUT the screens, and have Agent_SubmitLAForApproval call it. Leave the original screen
flow untouched.

=== PHASES (one at a time, tests + deploy + my OK between each) ===
  Phase 1: Read-only actions (ResolveAccount, GetAccountInsight, DescribeObject) + confirm
           all field API names. Deploy and demo.
  Phase 2: Preview/commit for Opportunity, then Order, then Loading Advice + Line.
           Tests must cover happy path, every missing-field branch, and
           commit-without-confirmation rejection.
  Phase 3: Headless auto-approval flow + Agent_SubmitLAForApproval. Verify PDF, bell
           notification and email all fire.
  Phase 4: Conversation layer -- build/wire the Agentforce agent (or the LWC fallback) and
           place it on the utility bar and the Account record page.
  Phase 5: End-to-end test from both surfaces, >=75% org coverage, FLS/sharing review, and
           a handover doc listing any manual Agent Builder steps and licensing needs.

Begin with STEP 0 now and wait for my confirmation of the org before continuing.
```

Appendix B — Loading Advice field cheat-sheet

Captured from the Create Loading Advice screens. Claude Code must verify API names against the org in Phase 1; labels below are what the user sees.

Field (label)	Req?	Notes
Type of Load	Yes	Stock Transfer (Internal) | Commercial (Sale) — chosen first
Customer	Yes	Account lookup (pre-filled on Account page)
Customer Type	Yes	Picklist
Location	Yes	Text
Product	Yes	Product lookup / search
Quantity (MT)	Yes	Number
Price (N/MT)	Yes	Currency
LPO Number	No	Text
Delivery Mode	No	Picklist
Currency	No	Picklist
Transportation (N/MT)	No	Number
Payment Status	No	Text
Plant Location	No	Picklist
Payment Term	No	Picklist
Heating Charge (N/TRK)	No	Number
Sprayer Rate (N/day)	No	Number
No. of Sprayer days	No	Number

End of plan. Start Claude Code with the Appendix A prompt; it will ask you to confirm the org connection before it does anything else.